

SENIOR JAVA INTERVIEW PREP

Meeting **Scheduler** System Design

Patterns · SOLID · UML · Full Code · Q&A

5

Design Patterns

5

SOLID Principles

38

Slides

10

Interview Q&As

Use ← → arrow keys or the buttons to navigate

MINDSET FIRST

How to Approach System Design

Think of it like **building a house**. You don't start swinging a hammer. You first ask: "*How many rooms? How many floors? Who lives here?*" — then you draw a blueprint.

Step 1 — Ask Questions

Clarify requirements. What must it do? What is out of scope?

Step 2 — Find Entities

Identify the "nouns" — the things your system deals with.

Step 3 — Choose Patterns

Solve design problems using proven blueprints (patterns).

Step 4 — Apply SOLID

Write code that is easy to change, extend, and test.

🧠 REMEMBER THIS

In an interview: **think out loud**. They want to see HOW you think — not just the final answer. Say "I'm choosing X because Y, and the trade-off is Z."

STEP 1

Always Start with Requirements

Before writing one line of code, **ask 3–4 clarifying questions**. This is what separates a senior engineer from a junior engineer.

Questions to Ask

- How many users — 100 or 1 million?
- Should it handle recurring meetings?
- Do we need timezone support?
- Single company or multi-tenant?
- Any calendar integrations needed? (Google, Outlook)

Scope for This Design

- Single company, in-memory storage
- Business hours: Mon–Fri, 9 AM–5 PM
- All users in same timezone
- No recurring meetings (future scope)
- No REST API or UI layer

💡 WHY THIS MATTERS

If you skip this and design a globally distributed system when they wanted a simple tool for 50 employees — you wasted everyone's time and showed poor judgment.

REQUIREMENTS

Functional Requirements

Functional = What the system must *do*



Schedule a Meeting

Pick participants, auto-find a free slot for everyone, book a room



Send Notifications

Email + SMS when a meeting is created, cancelled, or updated



Cancel a Meeting

Only the organizer can cancel. All participants get notified.



View Calendar

See a user's upcoming and past meetings



Reschedule

Cancel old slot, find new slot, re-notify everyone



Undo Last Action

Reverse the last schedule or cancel operation

REQUIREMENTS

Non-Functional Requirements

Non-Functional = How well the system must do it (quality attributes)

No Double-Booking

If two people try to book the same room simultaneously, only one succeeds. Needs thread safety.

Extensible

Adding Slack notifications or a new scheduling algorithm should not change existing code.

Testable

Every piece must be testable in isolation using mocks. No hidden dependencies.

Maintainable

A new developer should be able to understand any class by reading just that class — not the whole codebase.

🏠 REAL-WORLD ANALOGY

Functional = "The house must have 3 bedrooms and 2 bathrooms." Non-Functional = "The house must survive earthquakes and be energy-efficient."

STEP 2

What is an Entity?

An Entity is a **"thing"** in your system that has its own data and can perform actions.

SIMPLE WAY TO FIND ENTITIES

Read your requirements and underline all the **nouns**. Each noun that has data AND behaviour is a candidate entity.

In a meeting scheduler, the nouns are:

User

Meeting

Room

TimeSlot

Calendar

CORE ENTITIES

Our 6 Core Entities



User

A person who can organize or attend meetings. Has a name, email, phone, and personal calendar.



TimeSlot

A start time + end time window. Knows how to check if two slots overlap. *Immutable* — once created, it never changes.



Room

A physical meeting room with a name and capacity. Tracks which time slots are already booked.



Meeting

The core object. Has a title, time slot, room, list of participants, and who organized it.



UserCalendar

Belongs to one User. Tracks all their meetings. Answers the question: "Is this user free at 2 PM?"



MeetingScheduler

The main coordinator. Orchestrates everything — finds slots, books rooms, fires notifications.

SPECIAL CONCEPT

TimeSlot — A Value Object

A **Value Object** is immutable (never changes) and has no "identity". Two TimeSlots with the same start/end are considered equal — like how ₹100 is ₹100 regardless of which specific note it is.

💡 WHY MAKE IT A SEPARATE CLASS?

The logic "**do two time slots overlap?**" is used by BOTH `UserCalendar` AND `Room`. If we don't have a `TimeSlot` class, we write this logic twice. That's the DRY (Don't Repeat Yourself) principle — write it once in `TimeSlot.overlaps()`, use it everywhere.

```
// The overlap formula — one method, used everywhere
public boolean overlaps(TimeSlot other) {
    // Slot A overlaps Slot B if A starts before B ends AND A ends after B starts
    return this.start.isBefore(other.end) && this.end.isAfter(other.start);
}

// Visual example:
// A: [====]
// B:  [====] → They overlap! (A starts before B ends, A ends after B starts)
//
// A: [====]
// B:      [====] → No overlap
```


CORE ENTITY

The Meeting Entity

This is the most important entity. It ties everything together.

Fields

- `id` — unique identifier (UUID)
- `title` — "Sprint Standup"
- `description` — optional details
- `timeSlot` — when it happens
- `participants` — list of Users
- `room` — where it happens
- `organizerId` — who created it
- `status` — SCHEDULED / CANCELLED

Key Design Decisions

- Created using the **Builder pattern** (next slides)
- Once built, most fields are **immutable** (no setters)
- Only `status` can change (cancel/complete)
- Participants list is **unmodifiable** after creation

Status Flow

SCHEDULED → CANCELLED

SCHEDULED → COMPLETED

CANCELLED → SCHEDULED (undo)

STEP 3

What Are Design Patterns?

Design patterns are **proven solutions** to problems that software developers face again and again. They are like **blueprints** — not finished code, but templates you apply to your specific problem.

REAL-WORLD ANALOGY

When an architect designs a door, they don't reinvent how hinges work. They use a proven hinge design. Similarly, when we face a specific coding problem, we use a proven pattern.

We use 5 Patterns

-  **Builder** — creating complex objects
-  **Observer** — notifications
-  **Strategy** — algorithm selection
-  **Command** — undo/redo operations
-  **Singleton** — one instance only

Important Rule

Never force a pattern onto code just to "use patterns". First identify the **problem**, then pick the pattern that solves that **exact problem**.

Builder Pattern: The Problem

✗ THE PROBLEM — "TELESCOPING CONSTRUCTOR"

Meeting has 7+ fields. A normal constructor looks like this monstrosity:

```
// PROBLEM: Which parameter is which? Easy to swap title and description!
Meeting m = new Meeting(
    "Sprint Standup",      // title? or description?
    "Daily sync",         // description? or organizerId?
    timeSlot,             // ok
    room,                 // ok
    participantsList,     // ok
    "user-123",           // organizerId
    MeetingStatus.SCHEDULED // status
);
```

● WHY THIS IS BAD

If you swap two String parameters, the compiler won't complain — you'll only find the bug at runtime. Also, what if description is optional? You'd need 5 different constructors (telescoping).

Builder Pattern: The Solution

🍔 REAL-WORLD ANALOGY — CUSTOM BURGER ORDER

When you order a custom burger, you say: "Add cheese. Add lettuce. No onions. Done." Each step is clear. You can skip optional toppings. That's Builder — you add what you need, in any order, and call `build()` at the end.

✅ THE SOLUTION — BUILDER PATTERN

Create a `Builder` inner class. Each method sets one field and returns `this` (so you can chain). The `build()` method validates everything and creates the Meeting.

```
// SOLUTION: Self-documenting, readable, validated
Meeting meeting = new Meeting.Builder()
    .title("Sprint Standup")
    .description("Daily sync")    // optional - can skip
    .timeSlot(slot)
    .participants(List.of(alice, bob))
    .room(boardRoom)
    .organizerId(alice.getId())
    .build();                    // validates + creates Meeting
```

Now it's impossible to swap `title` and `description` — they have named methods!

Builder Pattern: The Code

```
public class Meeting {
    private final String id, title, description, organizerId;
    private final TimeSlot timeSlot;
    private final List<User> participants;
    private final Room room;

    private Meeting(Builder b) { // ← private! only Builder can call this
        this.id = UUID.randomUUID().toString();
        this.title = b.title;
        this.participants = Collections.unmodifiableList(b.participants);
        // ... rest of fields
    }

    public static class Builder { // ← static inner class
        private String title;
        private List<User> participants = new ArrayList<>();

        public Builder title(String t) { this.title = t; return this; }
        public Builder participants(List<User> p) { this.participants = p; return this; }
        // ... other fluent setters

        public Meeting build() {
            if (title == null) throw new IllegalStateException("Title required");
            return new Meeting(this); // ← calls private constructor
        }
    }
}
```

Benefits

Readable • Validated at one place • Immutable after build • Optional fields are easy

SOLID Connection

SRP — Builder is solely responsible for constructing Meeting correctly. OCP — add new fields without breaking existing code.

Observer Pattern: The Problem

✗ THE PROBLEM — HARD-CODED NOTIFICATIONS

When a meeting is scheduled, we need to notify people via Email AND SMS AND (in future) Slack. Without Observer, the scheduler knows about every channel:

```
public void scheduleMeeting(...) {  
    // ... booking logic ...  
  
    // PROBLEM: Scheduler is coupled to every notification channel!  
    emailService.send(participants, meeting);  
    smsService.send(participants, meeting);  
    slackService.send(participants, meeting); // adding Slack = modify this class!  
    pushService.send(participants, meeting); // adding push = modify again!  
    // This violates OCP - you must modify existing code to add features  
}
```

● WHY THIS IS BAD

Every time you add a new notification channel, you must modify the `scheduleMeeting()` method. That's risky — you might break existing functionality. You're also coupling unrelated concerns together.

Observer Pattern: The Solution

REAL-WORLD ANALOGY — YOUTUBE SUBSCRIPTIONS

When a YouTuber posts a video, they don't call each subscriber individually. Subscribers **register** to the channel. YouTube **notifies all subscribers** automatically. The YouTuber doesn't know or care who subscribed — that's Observer.

THE SOLUTION

Define a `MeetingEventListener` interface. Each notification service **implements** it and **registers** with the scheduler. When a meeting is scheduled, the scheduler calls all registered listeners. Adding Slack = write one class + register it. Zero changes to the scheduler.

Without Observer (tight coupling)

Scheduler → Email, SMS, Slack (scheduler knows all)

With Observer (loose coupling)

Scheduler → fires event → Email registers itself, SMS registers itself, Slack registers itself

Observer Pattern: The Code

```
// Step 1: Define the Observer interface (the contract)
public interface MeetingEventListener {
    void onMeetingScheduled(Meeting meeting);
    void onMeetingCancelled(Meeting meeting);
}

// Step 2: Each notification service implements the interface
public class EmailNotificationService implements MeetingEventListener {
    @Override
    public void onMeetingScheduled(Meeting m) {
        m.getParticipants().forEach(u -> sendEmail(u.getEmail(), m.getTitle()));
    }
    @Override
    public void onMeetingCancelled(Meeting m) { /* send cancellation email */ }
}

// Step 3: Scheduler holds a list of listeners and fires events
public class MeetingScheduler {
    private List<MeetingEventListener> listeners = new ArrayList<>();

    public void addListener(MeetingEventListener l) { listeners.add(l); }

    public Meeting scheduleMeeting(...) {
        // ... booking logic ...
        listeners.forEach(l -> l.onMeetingScheduled(meeting)); // fire event
        return meeting;
    }
}

// Step 4: Register listeners at startup – ONE LINE per channel
scheduler.addListener(new EmailNotificationService());
scheduler.addListener(new SMSNotificationService());
scheduler.addListener(new SlackNotificationService()); // ← zero code change in scheduler
```


Strategy Pattern: Swap Algorithms

📖 REAL-WORLD ANALOGY — GPS NAVIGATION

Google Maps lets you choose: Fastest Route, Avoid Tolls, or Walking. The destination is the same — only the **algorithm to get there** changes. That's Strategy — swap the algorithm without changing the rest of the app.

✗ PROBLEM WITHOUT STRATEGY

The scheduling algorithm is baked into the scheduler. Adding "prefer morning slots" or "AI-powered scheduling" means modifying core code.

✅ SOLUTION WITH STRATEGY

Define a `SchedulingStrategy` interface with one method: `findAvailableSlot()`. Create different implementations. Swap them with one line: `scheduler.setStrategy(new MorningPreferenceStrategy())`

```
public interface SchedulingStrategy {
    Optional<TimeSlot> findAvailableSlot(List<User> p, Duration d, LocalDateTime from);
}
class EarliestSlotStrategy implements SchedulingStrategy { /* checks every 30 min */ }
class MorningPreferenceStrategy implements SchedulingStrategy { /* prefers 9-12 AM */ }
```

Command Pattern: Undo & Audit

REAL-WORLD ANALOGY — TV REMOTE WITH UNDO

Every button press on the remote is a "Command" object. The TV has a history of commands. Pressing Undo on a universal remote reverses the last command. Each command knows exactly how to undo itself.

✗ WITHOUT COMMAND

Scheduling calls 4 methods: save, update calendars, book room, notify. To undo, you'd need special "undo code" scattered everywhere.

✓ WITH COMMAND

Wrap all 4 steps into a `ScheduleMeetingCommand`. It has `execute()` and `undo()`. Push to history stack. Undo = pop + call `undo()`.

```
interface MeetingCommand {
    void execute();
    void undo();
}

class ScheduleCmd implements MeetingCommand {
    public void execute() {
        repo.save(meeting);
        room.book(slot);
        // notify observers
    }
    public void undo() {
        repo.delete(meeting.getId());
        room.unbook(slot);
        // notify cancellation
    }
}

// Usage:
// cmd.execute(); history.push(cmd);
// history.pop().undo(); ← undo!
```

Singleton Pattern: One Instance Only

REAL-WORLD ANALOGY — COMPANY CEO

A company has exactly ONE CEO. Everyone in the company talks to the same CEO — not to a new CEO every time they need something. The Singleton ensures there is exactly one instance of a class in the entire application.

✓ WHY SINGLETON FOR MEETINGSCHEDULER?

The scheduler owns: the command history, the listeners list, and the strategy. If you create two instances, they each have their own history — undo wouldn't work correctly.

```
public class MeetingScheduler {  
  
    // volatile = all threads see same value  
    private static volatile MeetingScheduler instance;  
  
    private MeetingScheduler() {} // private!  
  
    public static MeetingScheduler getInstance() {  
        if (instance == null) {  
            synchronized (MeetingScheduler.class) {  
                if (instance == null) // double-check  
                    instance = new MeetingScheduler();  
            }  
        }  
        return instance;  
    }  
}
```

Senior Note — Tell the interviewer this!

In Spring Boot, you'd use `@Service` (singleton by default) instead. Implementing Singleton manually makes unit testing harder. I'm showing it here to demonstrate the concept.

STEP 4

What is SOLID?

SOLID is a set of **5 principles** that make code easier to **maintain, extend, and test**.
Think of it as "the rules of clean architecture."

S — Single Responsibility

One class = one reason to change

O — Open / Closed

Open for extension, closed for modification

L — Liskov Substitution

Subtypes must work where base types are expected

I — Interface Segregation

Don't force classes to implement what they don't need

D — Dependency Inversion

Depend on abstractions, not concrete classes

REMEMBER THIS

Don't just say the definition. In the interview, point to your code and say: *"Here's where SRP is maintained — look at this class. The only reason it would ever change is X."*

Single Responsibility Principle

"A class should have only one reason to change."

ANALOGY — RESTAURANT KITCHEN

The chef cooks. The waiter serves. The cashier bills. If the chef also handled billing and serving, changing the billing system would mean modifying the chef's job description — that's SRP violation.



Meeting

— only holds meeting data. No notification code, no scheduling algorithm.



EmailNotificationService

— only sends emails. It changes only if email format/provider changes.



UserCalendar

— only manages one user's schedule. No room-booking, no notification logic.



EarliestSlotStrategy

— only finds available slots. No room booking, no meeting creation.



SOLID — O

Open / Closed Principle

"Open for extension, closed for modification."

ANALOGY — POWER STRIP

A power strip lets you plug in new devices (extension) without rewiring your house walls (modification). OCP means your existing code is the "wall" — you extend it by adding new plugs, not by tearing it apart.

Adding Slack Notifications

Write `SlackNotificationService` implements `MeetingEventListener`. Call `addListener(slack)`. **Zero changes to existing classes.**

Adding AI Scheduling

Write `AI SchedulingStrategy` implements `SchedulingStrategy`. Call `setStrategy(ai)`. **Zero changes to existing classes.**

Switching to Database

Write `JpaMeetingRepository` implements `MeetingRepository`. Inject it. **Zero changes to existing classes.**

Adding Recurring Meetings

Extend `Meeting` with `RecurringMeeting`. Add `RecurringScheduleCommand`. **Zero changes to existing classes.**

Liskov Substitution Principle

"If A extends B, then wherever B is used, A should work too — without breaking anything."

CLASSIC ANALOGY — DUCK TEST

If it walks like a duck and quacks like a duck, it should be usable as a duck. If a `RubberDuck` extends `Duck` but can't swim — it violates LSP, because swimming is part of the Duck contract.



EmailNotificationService

replaces

MeetingEventListener

Anywhere the interface is used, EmailNotificationService works perfectly. Same contract, no surprises.



EarliestSlotStrategy

replaces

SchedulingStrategy

Returns the same type (`Optional<TimeSlot>`), never throws unexpected exceptions.



InMemoryMeetingRepository

replaces

MeetingRepository

Swap it for `JpaMeetingRepository` — all calling code works identically.

SOLID — I

Interface Segregation Principle

"Don't force a class to implement methods it doesn't need."

ANALOGY — JOB DESCRIPTION

If you hire a driver, you don't put "cook meals" and "write software" in their job description. Each interface should be focused — contain only what the implementor actually needs to do.

Good — Focused Interfaces

- `MeetingEventListener` : 3 methods only
- `SchedulingStrategy` : 1 method only
- `MeetingCommand` : 2 methods only
- `MeetingRepository` : 4 CRUD methods

Bad — Fat Interface

- One giant `IMeetingSystem` interface with 20 methods
- Every implementor must provide all 20 even if it uses 3
- Changing one method forces all implementors to change

Dependency Inversion Principle

"High-level modules should depend on abstractions (interfaces), not on concrete classes."

📌 ANALOGY — ELECTRICAL OUTLETS

Your phone charger depends on the standard electrical outlet **interface** (voltage, plug shape) — not on a specific power plant. You can switch power plants without changing your charger. DIP works the same way.



MeetingScheduler depends on SchedulingStrategy (interface)

Not on `EarliestSlotStrategy`. Swap algorithms with zero code change in the scheduler.



MeetingScheduler depends on MeetingRepository (interface)

Not on `InMemoryMeetingRepository`. Switch to database — scheduler doesn't know or care.



Commands depend on MeetingEventListener (interface)

Not on `EmailNotificationService`. Email, SMS, Slack — all treated identically.

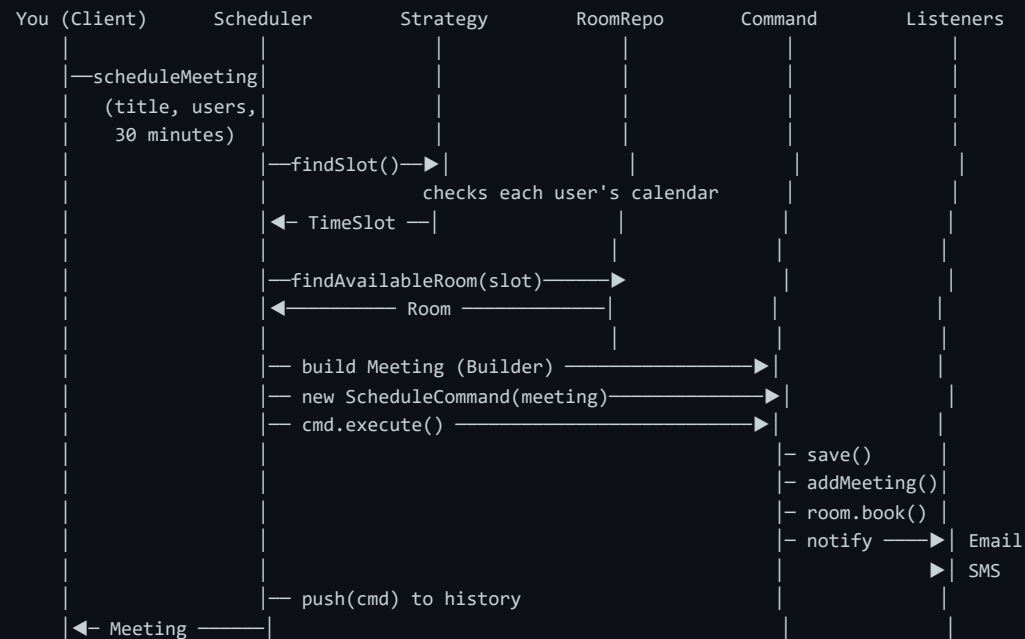
Practical benefit: In unit tests, you can mock the interface. You don't need a real email server to test the scheduler.

Class Diagram — How Everything Connects



◇ = composition (owns it) | ▷ = implements/depends on | --▷ = uses

What Happens When You Schedule a Meeting?



💡 KEY INSIGHT

Each step is done by a different class — this is SRP in action. The Scheduler *coordinates* but doesn't *do* any of the actual work itself.

TimeSlot & UserCalendar

```
public final class TimeSlot {           // final = cannot be subclassed
    private final LocalDateTime start, end;

    public TimeSlot(LocalDateTime s, LocalDateTime e) {
        if (!s.isBefore(e)) throw new IllegalArgumentException("Start must be before end");
        this.start = s; this.end = e;
    }

    public boolean overlaps(TimeSlot other) {
        return this.start.isBefore(other.end) && this.end.isAfter(other.start);
    }

    public LocalDateTime getStart() { return start; }
    public LocalDateTime getEnd()   { return end; }
}

public class UserCalendar {
    private final List<Meeting> meetings = new ArrayList<>();

    // Is the user free during this time slot?
    public boolean isAvailable(TimeSlot requested) {
        return meetings.stream()
            .filter(m -> m.getStatus() == MeetingStatus.SCHEDULED)
            .map(Meeting::getTimeSlot)
            .noneMatch(slot -> slot.overlaps(requested)); // ← reuses TimeSlot.overlaps()
    }

    public void addMeeting(Meeting m) { meetings.add(m); }
    public void removeMeeting(Meeting m) { meetings.remove(m); }
}
```

Meeting with Builder Pattern

```
public class Meeting {
    private final String id, title, organizerId;
    private final TimeSlot timeSlot;
    private final List<User> participants; // unmodifiable
    private final Room room;
    private MeetingStatus status = MeetingStatus.SCHEDULED;

    private Meeting(Builder b) { // only Builder can call this
        this.id = UUID.randomUUID().toString();
        this.title = b.title;
        this.timeSlot = b.timeSlot;
        this.participants = Collections.unmodifiableList(b.participants);
        this.room = b.room;
        this.organizerId = b.organizerId;
    }

    public void cancel() { status = MeetingStatus.CANCELLED; }

    public static class Builder {
        private String title, organizerId;
        private TimeSlot timeSlot;
        private List<User> participants;
        private Room room;

        public Builder title(String t) { this.title = t; return this; }
        public Builder timeSlot(TimeSlot ts) { this.timeSlot = ts; return this; }
        public Builder participants(List<User> p) { this.participants = p; return this; }
        public Builder room(Room r) { this.room = r; return this; }
        public Builder organizerId(String id) { this.organizerId = id; return this; }

        public Meeting build() {
            if (title == null) throw new IllegalStateException("Title required");
            return new Meeting(this); // gate: validates then constructs
        }
    }
}
```

Observer + Strategy Implementation

```
// — Observer Interface —————
public interface MeetingEventListener {
    void onMeetingScheduled(Meeting m);
    void onMeetingCancelled(Meeting m);
}

// — Email Notification (Observer #1) —————
public class EmailNotificationService implements MeetingEventListener {
    @Override
    public void onMeetingScheduled(Meeting m) {
        m.getParticipants().forEach(u ->
            System.out.printf("[EMAIL] %s → Meeting: %s at %s%n",
                u.getEmail(), m.getTitle(), m.getTimeSlot().getStart()));
    }
    @Override
    public void onMeetingCancelled(Meeting m) {
        m.getParticipants().forEach(u ->
            System.out.printf("[EMAIL] %s → CANCELLED: %s%n", u.getEmail(), m.getTitle()));
    }
}

// — Strategy Interface —————
public interface SchedulingStrategy {
    Optional<TimeSlot> findAvailableSlot(
        List<User> participants, Duration duration, LocalDateTime from);
}

// — Earliest Slot Strategy —————
public class EarliestAvailableSlotStrategy implements SchedulingStrategy {
    @Override
    public Optional<TimeSlot> findAvailableSlot(
        List<User> participants, Duration duration, LocalDateTime from) {

        LocalDateTime current = from;
        LocalDateTime limit = from.plusDays(14);

        while (current.isBefore(limit)) {
            current = skipToBusinessHours(current);
            TimeSlot candidate = new TimeSlot(current, current.plus(duration));
        }
    }
}
```

Command Pattern & MeetingScheduler

```
// — Command: encapsulates the entire schedule workflow —————
class ScheduleMeetingCommand implements MeetingCommand {
    private final Meeting meeting;
    private final MeetingRepository repo;
    private final List<MeetingEventListener> listeners;

    @Override
    public void execute() {
        repo.save(meeting); // 1. persist
        meeting.getParticipants().forEach(u -> u.getCalendar().addMeeting(meeting)); // 2. update calendars
        meeting.getRoom().book(meeting.getTimeSlot()); // 3. reserve room
        listeners.forEach(l -> l.onMeetingScheduled(meeting)); // 4. notify observers
    }

    @Override
    public void undo() { // reverses all 4 steps
        meeting.cancel();
        meeting.getRoom().unbook(meeting.getTimeSlot());
        meeting.getParticipants().forEach(u -> u.getCalendar().removeMeeting(meeting));
        repo.delete(meeting.getId());
        listeners.forEach(l -> l.onMeetingCancelled(meeting));
    }
}

// — Singleton Scheduler: orchestrates everything —————
public class MeetingScheduler {
    private static volatile MeetingScheduler instance;
    private final List<MeetingEventListener> listeners = new ArrayList<>();
    private final Deque<MeetingCommand> history = new ArrayDeque<>();
    private SchedulingStrategy strategy = new EarliestAvailableSlotStrategy();

    public Meeting scheduleMeeting(String title, List<User> participants, Duration d, String orgId) {
        TimeSlot slot = strategy.findAvailableSlot(participants, d, LocalDateTime.now()).orElseThrow();
        Room room = roomRepo.findAvailableRoom(slot, participants.size()).orElseThrow();
        Meeting m = new Meeting.Builder().title(title).timeSlot(slot).participants(participants).room(room).organizerId(orgId).build();
        MeetingCommand cmd = new ScheduleMeetingCommand(m, meetingRepo, listeners);
        cmd.execute(); history.push(cmd);
        return m;
    }
}
```

Putting It All Together — Main.java

```
public class Main {  
    public static void main(String[] args) {  
  
        // — 1. Get the scheduler (Singleton) —————  
        MeetingScheduler scheduler = MeetingScheduler.getInstance();  
  
        // — 2. Add rooms —————  
        scheduler.addRoom(new Room("R1", "Board Room", 10));  
        scheduler.addRoom(new Room("R2", "Zoom Room", 4));  
  
        // — 3. Register notification observers —————  
        scheduler.addListener(new EmailNotificationService());  
        scheduler.addListener(new SMSNotificationService());  
  
        // — 4. Create users —————  
        User alice = new User("u1", "Alice", "alice@co.com", "555-0101");  
        User bob   = new User("u2", "Bob",   "bob@co.com",   "555-0102");  
  
        // — 5. Schedule a meeting - auto-finds free slot + room —  
        Meeting standup = scheduler.scheduleMeeting(  
            "Daily Standup", List.of(alice, bob), Duration.ofMinutes(30), alice.getId()  
        );  
        System.out.println("Booked: " + standup.getTitle() + " @ " + standup.getTimeSlot().getStart());  
  
        // — 6. Cancel it —————  
        scheduler.cancelMeeting(standup.getId(), alice.getId());  
        System.out.println("Status: " + standup.getStatus()); // CANCELLED  
  
        // — 7. Undo the cancel (Command pattern!) —————  
        scheduler.undoLastAction();  
        System.out.println("After undo: " + standup.getStatus()); // SCHEDULED again  
  
        // — 8. Swap algorithm at runtime (Strategy pattern!) —————  
        scheduler.setStrategy(new MorningPreferenceStrategy());  
    }  
}
```


EXTENSIBILITY

Adding Features — Zero Code Changes

This table shows how clean design pays off. Every new feature is additive — not invasive.



Slack Notifications

Create `SlackListener` implements `MeetingEventListener` → `scheduler.addListener(slack)` . Done.



AI Scheduling

Create `AISchedulingStrategy` implements `SchedulingStrategy` → `setStrategy(ai)` . Done.



Database Storage

Create `JpaMeetingRepository` implements `MeetingRepository` . Inject into scheduler. Done.



Recurring Meetings

Create `RecurringMeeting` extends `Meeting` + `RecurringScheduleCommand` . Existing code untouched.



Timezone Support

Change `TimeSlot` to use `ZonedDateTime` . Since overlap logic lives only there, everything else auto-works.

How to Present This in 45–60 Minutes

0–5 min

Ask Clarifying Questions

"Before I start — can I ask a few questions to make sure I'm solving the right problem?" Then ask 3–4 questions. Write confirmed requirements on the board.

5–10 min

List Core Entities

Say: "The nouns in our system are User, Meeting, Room, TimeSlot, UserCalendar. I'll explain why TimeSlot is a Value Object..."

10–20 min

Walk Through Design Patterns

For each: 1) State the problem, 2) State the pattern, 3) Show how it solves it. Use analogies. Keep it conversational.

20–28 min

SOLID Principles

Point to specific classes. "Look at EmailNotificationService — the only reason it would change is if the email format changes. That's SRP."

28–38 min

Draw UML + Walk Code

Draw the class diagram. Then walk through the sequence: "When scheduleMeeting() is called, here's what happens step by step..."

38–45 min

Extensibility + Trade-offs

Show 2–3 extension scenarios. Then proactively mention limits: "In production, I'd use Spring @Service instead of manual Singleton."

45–60 min

Q&A

Be ready for deep-dives. Always answer with trade-offs. See next slides.

INTERVIEW TIPS

Phrases That Signal Senior-Level Thinking

Use These Phrases

- "Before I start, let me clarify..."
- "The problem without this pattern is..."
- "I'm making a trade-off here — X gives us Y at the cost of Z"
- "This follows OCP because adding a new feature means writing a new class, not modifying existing code"
- "In production, I would use Spring's IoC instead because..."
- "The failure mode here is... and I'd handle it by..."

Avoid These

- "I used Builder because it's a pattern" (say WHY)
- Reciting SOLID definitions without pointing to code
- Saying "it scales" without explaining how
- Going silent — think out loud always
- Defending a bad choice — it's OK to say "this has a trade-off"

THE GOLDEN RULE

Show your **reasoning process**, not just the answer. A senior dev who gets a wrong answer but shows excellent reasoning is more impressive than someone who gives the right answer with no explanation.

Questions a Senior Dev Will Ask You

Q1. How do you prevent two people from booking the same room at the same time?

Your answer: The `Room.book()` method is `synchronized`. Only one thread enters it at a time — the check and the booking happen atomically. In production with a DB, I'd use optimistic locking (`@Version`) — saves lock overhead when conflicts are rare, retries on the rare collision.

Q2. Why Singleton? Isn't that an anti-pattern?

Your answer: For this design, yes — the scheduler needs to coordinate shared state. But I'd proactively say: "In Spring Boot, I'd use `@Service` (singleton-scoped by default) with constructor injection — much more testable. I'm using manual Singleton here to demonstrate the concept."

Q3. How would you test this system?

Your answer: Unit test each class independently: `TimeSlot.overlaps()` with 6 edge cases. Test `EarliestSlotStrategy` with mocked user calendars. For the scheduler, mock the `SchedulingStrategy`, `MeetingRepository`, and listeners — that's why we depend on interfaces (DIP). Integration tests use the InMemory implementations — no mocking needed.

Questions a Senior Dev Will Ask You

Q4. How do you handle timezones?

Your answer: Currently `TimeSlot` uses `LocalDateTime` — no timezone. To fix: use `ZonedDateTime` internally. Store everything in UTC, display in user's local timezone. Since overlap logic lives only in `TimeSlot.overlaps()` (DRY), the rest of the code doesn't change.

Q5. How do you scale this to 1 million users?

Your answer: 1) Swap `InMemoryMeetingRepository` for `JpaMeetingRepository` (PostgreSQL). 2) Cache availability lookups in Redis — reads are far more frequent than writes. 3) Move notifications to a message queue (Kafka) — don't block the booking on email sending. 4) Shard by organization ID. The clean architecture means only the infrastructure layer changes — not the business logic.

Q6. What if email sending fails midway through notifications?

Your answer: Currently it would bubble up and could affect the flow. Fix: use the **Outbox Pattern** — write the event to a DB table in the same transaction as the booking. A separate process reads and delivers notifications. Guarantees at-least-once delivery without blocking the booking.

Q7. Why Command instead of just calling methods directly?

Your answer: Direct method calls lose history. Command gives you: 1) Undo/redo for free, 2) Audit trail (serialize commands to a log table), 3) Queueing (process later), 4) Composable operations. Trade-off: more classes. Worth it when undo or audit is required.

Questions a Senior Dev Will Ask You

Q8. How would you add recurring meetings?

Your answer: Create a `RecurrenceRule` class (DAILY/WEEKLY/CUSTOM + end date). Create `RecurringMeeting` extends `Meeting`. Add a `ScheduleRecurringCommand` that generates individual meetings per occurrence. To cancel all: cancel the parent, cascade to children.

Q9. What's the time complexity of `findAvailableSlot()`?

Your answer: Current: $O(448 \times P \times M)$ — 14 days $\times$ 16 business hours $\times$ 2 slots/hour, times P participants, times M meetings each. Effectively $O(1)$ for real-world inputs. In DB version: an indexed SQL query on (userId, startTime) makes it $O(P \times \log N)$.

Q10. How would you make this distributed across multiple servers?

Your answer: The current Singleton breaks across JVMs. Fixes: 1) Shared DB for meeting/room records. 2) Distributed locking (Redis SETNX) instead of `synchronized`. 3) Kafka for event publishing instead of in-process Observer. 4) Store command history in DB as an audit/event table. Key insight: the clean boundaries (Repository, Observer, Command) make this migration surgical — not a rewrite.

🚩 YOU'RE READY!

You now have the requirements, entities, patterns, SOLID, UML, code, extensibility, presentation guide, and 10 Q&As. Go show them who's the senior developer!